

컴퓨터비전 응용 1주차 요약본

1. OpenCV 영상정규화 및 Object Detection 입문

1.1 디지털 영상 처리 개념

- 디지털 영상 처리 분류
 - 화소점 처리 (Point Processing)
 - 개별 픽셀을 조작하는 기법
 - 예: 명암 조정, 대비 조정
 - 영역 처리 (Area Processing)
 - 픽셀과 주변 픽셀을 함께 고려하여 처리
 - 예: 블러링, 엣지 검출
 - 기하학 처리 (Geometric Processing)
 - 화소의 위치를 변경하여 변환
 - 예: 이미지 회전, 확대/축소
 - 프레임 처리 (Frame Processing)
 - 여러 개의 영상을 조합하여 새로운 영상 생성
 - 예: 영상 합성

1.2 디지털 영상 신호 변환 과정

- 표본화 (Sampling)
 - 아날로그 신호를 이산적인 픽셀 단위로 변환하는 과정
→ 연속 신호에서 일정 간격으로 값만 뽑아 오는 과정
 - 영상에서는 "가로 × 세로 픽셀 수"에 해당
 - 샘플링 횟수가 많을수록 해상도가 증가함
- 양자화 (Quantization)

- 각 픽셀의 값을 특정 레벨로 변환하는 과정 → 샘플 값(연속값)을 **한정된 개수의 레벨**로 나누어 표현
 - 색상을 표현하는 비트 수가 많을수록 세밀한 색상 표현 가능
 - 영상에서는 보통 0~255(8bit) 밝기 단계에 매핑
 - **샘플링 레이트와 나이퀴스트 레이트**
 - **Sampling Rate(Hz)**
 - 단위 시간당 샘플 개수
 - 영상에서는 해상도/프레임레이트와 관련
 - **Nyquist Rate**
 - 정보 손실 없이 아날로그 신호를 복원하기 위한 최소 샘플링 속도
 - 너무 낮으면 계단·깨짐(aliasing)이 발생
- 즉, 샘플링/양자화를 잘못하면 **정보 손실**이 생기고, 이는 곧 화질 저하로 이어짐

1.3 영상 표현 방법

- **색공간 모델**
 - 색공간(Color Space)의 개념
 - 색을 숫자로 표현하기 위한 좌표계
 - 예: RGB, HSL, HSV
 - 작업 목적(검출·분할·표현)에 따라 **적합한 색공간을 선택**하는 것이 중요
- **RGB (Red, Green, Blue)**
 - 디스플레이 장치(모니터, 카메라)에서 기본으로 사용하는 색 표현
 - 각 채널 0~255 값 → (R,G,B) 조합으로 색 표현
 - **단점:** 조명 변화에 매우 민감 → 색 기반 분할에 불리
- **HSV (Hue, Saturation, Value)**
 - Hue(색상), Saturation(채도), Value(명도)
 - 인간의 직관적 색 인지에 더 가까운 표현

- **장점:** 조명 변화에 덜 민감, 특정 색상 영역(예: 파란 물체) 분할에 유리
- 실습에서는 주로 **RGB → HSV 변환 후, 색 기반 마스크 생성**에 많이 사용하게 될 내용

1.4 화소 연산 및 히스토그램 분석

• 화소 연산 (Pixel Arithmetic)

- 정의: 영상의 각 픽셀(0~255 값)에 대해 **수학적 연산**을 수행하는 것
- 목적: 밝기·대비·색상·필터링·합성 등 영상 특성을 조정
- 종류
 - 산술 연산: 덧셈, 뺄셈, 곱셈, 나눗셈
 - 논리 연산: AND, OR, XOR, NOT
 - 스칼라 연산: 픽셀 + 상수, 픽셀 × 상수
 - 마스크 연산: 특정 영역에만 연산 적용

• 덧셈 연산 `cv2.add`

- 개념: 두 영상 혹은 영상 + 상수를 더함
- 상수 더하기 → **영상 전체 밝기 증가**
- 두 영상 더하기 → 합성(블렌딩 전 단계)
- 특징: 포화(saturation) 연산 적용 → 255를 넘으면 255로 고정

• 뺄셈 연산 `cv2.subtract`

- 개념: 두 영상 혹은 영상 - 상수
- 상수 빼기 → 밝기 감소
- 두 영상 차이 → **변화/모션 검출**
- 특징: 0보다 작아지면 0으로 고정

• 가중합 연산 `cv2.addWeighted`

- 개념: **$result = \alpha \cdot img1 + \beta \cdot img2 + \gamma$**
- 용도: 두 영상을 비율대로 섞는 블렌딩, 트랜지션 효과

• 절대값 차이 `cv2.absdiff`

- 개념: 픽셀별 절대값 차이
- 용도: 변화 영역 강조, 모션 감지

• 마스크 연산

- 마스크: 단일 채널(흑백) uint8 이미지
- 0 → 적용하지 않음
- 0이 아닌 값(보통 255) → 적용
- 크기: 원본 영상의 H×W와 동일(컬러 영상이어도 마스크는 1채널)

역할	함수	특징 / 활용
마스크 생성	<code>cv2.inRange</code>	특정 색/값 범위로 마스크 생성 – 색 기반 분할
마스크 생성	<code>cv2.threshold</code> , <code>adaptiveThreshold</code>	전역/적응형 임계값으로 이진화 – 배경/전경 분리
마스크 생성	도형 그리기 함수들	직접 도형을 채워 ROI 정의
마스크 적용	<code>cv2.bitwise_and</code>	마스크 영역만 남기기 – ROI 추출 두 영상/마스크의 교집합 영역 만 보존
마스크 적용	<code>cv2.bitwise_or</code>	두 영상의 합집합 영역 강조
마스크 적용	<code>cv2.bitwise_xor</code>	서로 다른 영역만 강조(차이나는 부분)
마스크 적용	<code>cv2.bitwise_not</code>	마스크 영역 반전 영상/마스크 반전 (검→흰, 흰→검)
마스크 적용	<code>cv2.inpaint</code>	마스크 영역 복원 – 얼룩 제거
후처리	모폴로지 연산	노이즈 제거, 구멍 메우기
후처리	블러	경계 부드럽게, 날카로운 부분 정리

• 히스토그램 정규화

- 개념: 픽셀 값 분포를 일정한 범위(0~255) 전체에 걸쳐 **선형적으로 펼치는 작업**
- 목적: 영상의 밝기/대비 향상 및 특징 추출 전, 전처리 단계로 사용
- 원리: 최소값 → 0, 최대값 → 255에 맞추고, 나머지 값은 선형 변환

- 예) 원본: 중앙에 몰려 어둡고 대비 낮음, 정규화 후: 분포가 넓게 퍼지고, **대비가 더 뚜렷해짐**
- **히스토그램 평활화**
 - 개념: 픽셀 값 분포를 **가능한 균일하게** 되도록 재배치
 - 목적: 어두운/밝은 특정 영역에 몰려 있던 픽셀을 골고루 퍼뜨려 **디테일을 잘 보이게** 함
 - OpenCV 함수: `cv2.equalizeHist` (그레이스케일 기준)

1.5 이미지 읽기 및 표시

```
img = cv2.imread('sample.jpg') # 이미지 불러오기
cv2.imshow('Image', img) # 이미지를 창에 띄우기
cv2.waitKey(0) # 키 입력 대기
cv2.destroyAllWindows() # 모든 창 닫기
```

1.6 색공간 변환

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # 흑백 변환, BGR → GRAY
cv2.imshow('Gray Image', gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

1.7 이미지 크기 조정 및 회전

```
resized = cv2.resize(img, (300, 300)) # 이미 크기 조정
rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE) # 이미지 90도 회전
cv2.imshow('Resized', resized)
cv2.imshow('Rotated', rotated)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

2. OpenCV 를 이용한 기하학적 변환

2.1 필터란 무엇인가?

- 정의: 영상에서 각 픽셀 주변의 이웃 픽셀들을 고려하여 **새 픽셀 값을 계산**하는 연산
- 방식: 작은 행렬(커널, kernel)을 영상 위에 **슬라이딩(convolution)** 하면서 연산
- 목적
 - **Smoothing / Blurring**: 노이즈 제거, 부드럽게 만들기
 - **Sharpening / Edge Detection**: 경계(엣지) 강조
 - **특징 추출 전처리**: 디러닝, 코너 검출, 텍스처 분석 전에 사용
 - → “필터 = 커널을 이용해서 주변 정보들을 섞어 새 픽셀을 만드는 연산”

- **Smoothing / Blurring 계열 → 흐릿해지면서 노이즈가 줄어드는 모습**

목적	필터 종류	특징	OpenCV 함수
노이즈 제거 / 블러	평균 필터	커널 내 픽셀 평균값 으로 대체	<code>cv2.blur</code>
	가우시안 블러	가우시안 분포 기반 가중 평균, 자연스러운 블러	<code>cv2.GaussianBlur</code>
	중간값 필터	중앙값으로 대체 → salt & pepper 노이즈 에 강함	<code>cv2.medianBlur</code>
	양방향 필터	경계를 살리면서 블러링 (edge-preserving)	<code>cv2.bilateralFilter</code>

```
# 평균값 필터(5x5)
kernel = np.ones((5, 5), np.float32) / 25
blurred = cv2.filter2D(img, -1, kernel)

# 가우시안 블러(Gaussian Blur)
blurred = cv2.GaussianBlur(img, (5,5), 0)

# 양방향 필터(Bilateral Filter)
filtered = cv2.bilateralFilter(img, 9, 75, 75)
```

- **Sharpening / Edge Detection 계열 → 원본에서 경계만 남는 모습**

목적	필터	특징	OpenCV 함수
엣지 검출	소벨(Sobel)	1차 미분 기반, 방향성 있는 엣지 검출	<code>cv2.Sobel</code>
	라플라시안(Laplacian)	2차 미분 기반, 전체 엣지 강조	<code>cv2.Laplacian</code>
	캐니(Canny)	그래디언트 + 히스테리시스 임계값으로 정교한 엣지	<code>cv2.Canny</code>

```
# 샤프닝 필터(Sharpening Filter)
kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]])
sharpened = cv2.filter2D(img, -1, kernel)

# 엣지 검출 (Canny Edge Detection)
edges = cv2.Canny(img, 100, 200) # 엣지(경계선) 검출
cv2.imshow('Edges', edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

• Morphology(모폴로지) 연산 계열 → 노이즈가 사라지고 객체가 매끄러워

목적	연산	의미/효과	OpenCV 함수
노이즈 제거	침식(Erosion)	밝은 영역 수축 → 작은 밝은 점 (노이즈) 제거	<code>cv2.erode</code>
구멍 메우기, 연결	팽창(Dilation)	밝은 영역 확장 → 구멍 메움, 객체 연결	<code>cv2.dilate</code>
작은 노이즈 제거	열기(Opening)	침식 후 팽창	<code>cv2.morphologyEx(..., OPEN)</code>
작은 구멍 메우기	닫기(Closing)	팽창 후 침식	<code>cv2.morphologyEx(..., CLOSE)</code>

• 임의의 커널

- `cv2.filter2D`
 - 사용자가 직접 정의한 커널(예: 샤프닝 커널, 엣지 강조 커널 등)을 적용
 - 다양한 실험적 필터를 만들 때 활용

2.2 기하학적 변환 (Geometric Transformations)

- **개념:** 이미지에서 좌표 (x, y) 를 새로운 좌표 (x', y') 로 바꾸는 것
- 이 변환은 대부분 **행렬 곱으로 표현**할 수 있음 (2×3 , 3×3 변환 행렬 등)
- OpenCV에서 이미지 = **액자(Frame)** + **액자 안에 끼운 그림(Image)**

2.3 회전(Rotation), 확대/축소(Scaling)

• 회전(Rotation)

- **정의:** 지정한 기준점(보통 영상 중심)을 기준으로 영상 전체를 **일정 각도만큼 회전**
- **목적:** 물체 방향 정규화(기하학적 보정) 및 데이터 증강(augmentation)
- **OpenCV 함수**

- `cv2.getRotationMatrix2D(center, angle, scale)`

- `center` : 회전 중심
- `angle` : 각도(도 단위)
- `scale` : 크기 배율

```
# 이동 (Translation): 픽셀을 특정 방향으로 이동하는 변환
M = np.float32([[1, 0, 50], [0, 1, 30]]) # x축 50, y축 30 이동
translated = cv2.warpAffine(img, M, (width, height))
```

```
# 회전 (Rotation): 이미지를 특정 각도로 회전 (예: 45도)
M = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)
rotated = cv2.warpAffine(img, M, (cols, rows))
```

• 확대 / 축소 (Scaling)

- **정의:** 좌표를 일정 비율로 늘리거나 줄이는 변환
- **목적:** 해상도 변경/객체 크기 정규화/데이터 증강
- **문제점:** 픽셀을 당겨/줄이다 보니 **비는 공간이 생김** → **보간 필요**

```
# 확대 및 축소 (Scaling): fx, fy 값으로 크기 조절 (배율 조정 가능)
scaled = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_LINEAR)
```


◦ 보간(Interpolation) 방법

▪ Nearest Neighbor

- 가장 가까운 픽셀 값 복사
- 계산 빠르지만 계단 현상

```
# 최근접 이웃 보간법 (Nearest Neighbor)
resized = cv2.resize(img, (width*2, height*2),
                    interpolation=cv2.INTER_NEAREST)
# → 가장 가까운 픽셀 사용, 속도는 빠르지만 계단 현상(Aliasing)이 발생 가능

# 이웃 평균 보간법 (Neighbor Averaging)
resized = cv2.resize(img, (width*2, height*2), interpolation=cv2.INTER_
AREA)
# → 더 부드러운 결과를 얻을 수 있음
```

▪ Bilinear, Bicubic

- 주변 여러 픽셀을 이용한 보간
- 더 부드러운 결과, 계산량 증가

```
# 양선형 보간법 (Bilinear Interpolation)
resized = cv2.resize(img, (width*2, height*2), interpolation=cv2.INTER_
LINEAR)
# → 인접한 4개의 픽셀의 평균을 계산하여 새로운 픽셀 생성

# 고차 보간법 (Higher Order Interpolation)
resized = cv2.resize(img, (width*2, height*2), interpolation=cv2.INTER_
CUBIC)
# → 인접한 16개의 픽셀을 사용하여 더 부드러운 결과 제공
```

2.4 Affine 변환

- 개념: 2D 평면 상의 점을 "선형 변환(회전·축척·기울임) + 평행 이동"으로 바꾸는 변환
 - 직선 → 직선, 평행선 → 여전히 평행

→ 따라서 결과가 사다리꼴/평행사변형처럼 기울어진 형태가 됨

- 크기, 회전, 이동, 기울임(shear) 모두 **Affine 변환에 포함**
- 앞에서 배운 회전·이동·확대/축소도 수학적으로는 Affine의 특수한 경우

```
# 전단 변환 (Shearing): 이미지를 대각선으로 찌그러뜨리는 변환
```

```
M = np.float32([[1, 0.5, 0], [0.5, 1, 0]])
```

```
sheared = cv2.warpAffine(img, M, (width, height))
```

```
# Warping (고무판 변형): 이미지를 임의의 모양으로 변형 (비선형 변환 포함)
```

```
pts1 = np.float32([[50, 50], [200, 50], [50, 200]])
```

```
pts2 = np.float32([[10, 100], [200, 50], [100, 250]])
```

```
M = cv2.getAffineTransform(pts1, pts2)
```

```
warped = cv2.warpAffine(img, M, (cols, rows))
```

- **목적&활용**

- 문서 스캔 시 기울어진 이미지를 **반듯하게 정렬**
- 카메라가 약간 기울어진 경우 이미지 보정
- 특정 기준 좌표계로 맞춰야 하는 전처리 단계

- **OpenCV 함수**

- `cv2.getAffineTransform(pts1, pts2)`
 - 원본 영상의 3개 점, 변환 후 3개 점을 대응시켜 **2×3 행렬** 계산
- `cv2.warpAffine(src, M, dsize)`
 - 실제로 이미지를 변환할 때 사용

2.5 Perspective(투시) 변환

- **개념:** 카메라의 **원근 효과**를 반영하는 변환
 - 직선은 직선으로 유지되지만, 평행선이 **소실점에서 만남** → 평행성이 깨짐
- 예: 멀리 있는 철도가 한 점에서 만나는 것처럼 보이는 현상
- **활용**
 - 사다리꼴로 촬영된 문서를 **정사각형/직사각형**으로 펴기

- 도로/그라운드 Bird's-eye View(탑뷰) 생성
- 빌보드, 스크린 등에 이미지를 맞춰 붙이는 합성
- **OpenCV 함수**
 - `cv2.getPerspectiveTransform(pts1, pts2)`
 - 원본의 네 점, 결과 영상의 네 점을 대응시켜 **3×3 행렬(H)** 계산
 - `cv2.warpPerspective(src, M, dsize)`
 - 실제로 이미지에 투시 변환 적용

3. OpenCV를 이용한 동영상 처리, Object Tracking

3.1 영상의 이진화(Binarization)

- **이진화** : 디지털 이미지를 두 가지 값(0= 검정/ 255= 흰색)만으로 표현하는 방법
 - 배경과 객체를 명확히 구분해야 하는 경우 사용
 - ex) 번호판 인식, 문서스캔, 마스크 생성 등
- **전역 임계값(Thresholding)**
 - 개념
 - 이미지 전체에 하나의 임계값 T를 적용
 - 픽셀 값이 T이상이면 255, 아니면 0
 - 특징
 - 단순하고 매우 빠름
 - 조명 변화가 작은 이미지에서 효과적
 - 조명이 심하게 다른 경우에는 안정적이지 않음
 - 코드 예시

```
ret, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
```

- **적응형 임계값(Adaptive Thresholding)**

- 개념

- 이미지 내 픽셀의 국소영역(Local Area)마다 다른 임계값을 적용
- 조명 변화가 심한 이미지에서 매우 효과적

- 종류

- Mean Adaptive : 주변 영역 평균값 기준
- Gaussian Adaptive : 가우시안 가중 평균값 기준

- 장점

- 조명 변화에 강함
- 문서 스캔, 손글씨 인식 등에서 자주 사용

- 코드 예시

```
binary = cv2.adaptiveThreshold(  
    gray, 255,  
    cv2.ADAPTIVE_THRESH_MEAN_C,  
    cv2.THRESH_BINARY,  
    11, 2  
)
```

3.2 영상의 미분과 엣지 검출

- 영상과 1차 미분

- 미분(Gradient): 픽셀 간 밝기의 변화량을 측정하여 경계를 검출하는 기법
 - 변화가 크면 = 경계(Edge)
 - 변화가 작으면 = 평탄한 영역
- 1차 미분을 수행하면 영상의 경계 정보를 추출할 수 있음(Edge Detection)

- Sobel 필터 (Sobel Edge Detection)

- 영상의 경계 검출(경계 방향과 세기를 계산가능)
- 이미지에서 x, y 방향의 그래디언트(변화량) 계산

```
# OpenCV에서 Sobel 필터 적용
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3) # x 방향 미분
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3) # y 방향 미분
sobel_combined = cv2.magnitude(sobel_x, sobel_y) # 그래디언트 크기 계산
```

3.3 Canny 엣지 검출 (Canny Edge Detection)

- Canny 엣지 검출 과정

1. 가우시안 블러 적용 (노이즈 제거)

- 엣지는 노이즈에 매우 민감하여 먼저 블러링

2. 밝기변화(Gradient) 계산

- Sobel 필터를 사용하여 경계의 방향과 세기 계산

3. 비최대 억제 (Non-Maximum Suppression)

- 경계선을 얇게 만드는 과정 → 엣지만 남길 잡음 제거

4. 이중 임계값(Double Threshold)

- 강한 엣지(Hight threshold 이상) : 확실한 엣지
- 약한 엣지(Low~High) : 후보
- 낮은 값 미만 : 제거

5. 히스테리시스(Hysteresis) Edge Tracking

- 약한 엣지가 강한 엣지와 연결된 경우만 엣지로 인정

```
# OpenCV에서 Canny 엣지 검출 적용
edges = cv2.Canny(img, 100, 200) # 100, 200 → 각각 하한, 상한 임계값
```

3.3 허프 변환 (Hough Transform)

- 허프 변환은 이미지에서 직선 또는 원을 검출하는 알고리즘

```
# OpenCV에서 허프 직선 검출
lines = cv2.HoughLines(edges, 1, np.pi / 180, 150)
```

```

for rho, theta in lines[:, 0]:
    a, b = np.cos(theta), np.sin(theta)
    x0, y0 = a * rho, b * rho
    x1, y1 = int(x0 + 1000 * (-b)), int(y0 + 1000 * (a))
    x2, y2 = int(x0 - 1000 * (-b)), int(y0 - 1000 * (a))
    cv2.line(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

# 허프 변환을 활용한 원 검출
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, 1, 20,
                           param1=50, param2=30, minRadius=10, maxRadius=100)
for circle in circles[0, :]:
    cv2.circle(img, (circle[0], circle[1]), circle[2], (0, 255, 0), 2)
    # cv2.circle()을 사용하여 검출된 원을 영상에 그리기

```

3.4 객체 검출 및 분석

- 연결 요소 분석 (레이블링)

- 각각의 객체는 고유한 레이블을 부여 받음

```
ret, labels = cv2.connectedComponents(binary_img)
```

- 외곽선 검출(Contours)

```

contours, _ = cv2.findContours(binary_img,
                               cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(img, contours, -1, (0, 255, 0), 2)
# cv2.drawContours()를 사용하여 검출된 외곽선을 영상에 표시

```

- 기하학적 모멘트 (Geometric Moments)

- 객체의 형상을 수치적으로 분석하는 기법
- 객체의 픽셀 값 분포를 기반으로 중심점, 면적, 모양 특징을 계산하는 데 사용됨

```

# 중심점 추출
M = cv2.moments(contour) # 객체의 중심점 계산

```

```
cx = int(M["m10"] / M["m00"])
cy = int(M["m01"] / M["m00"])
```

- **Hu's 불변 모멘트 (Hu's Invariant Moments)**

- 위치변화, 회전, 크기변화에도 변하지 않는 불변의 특징 → 객체 인식에 유용

```
# 객체의 특징을 불변값으로 표현
hu_moments = cv2.HuMoments(cv2.moments(contour)).flatten()
```

→ 다른 객체와의 유사성을 비교하는 데 활용

3.5 배경 차분 (Background Subtraction)

- **기본 배경 차**

- 움직이는 객체와 배경을 자동으로 분리
- 실시간 영상 처리에서 자주 사용

```
fgbg = cv2.createBackgroundSubtractorMOG2()
fgmask = fgbg.apply(frame)
```

- **이동 평균 배경 차분**

- 더 단순한 방식으로 배경이 천천히 변화할 때 유리

```
# 이전 프레임과 현재 프레임을 가중 평균하여 배경을 업데이트
alpha = 0.05
bg = cv2.addWeighted(bg, 1 - alpha, frame, alpha, 0)
fgmask = cv2.absdiff(bg.astype(np.uint8), frame) # 움직이는 객체 검출
```

4. 추적 알고리즘 실습

4.1 Optical Flow

- 개념 : 두 연속 프레임 사이에서 픽셀의 이동방향과 크기를 추정하는 기술

- 픽셀의 이동, 방향, 속도 정보
- 영상 내 객체 이동 분석에 필수
- 사용 예시) 사람/물체 이동 추적, 차량 상대속도 측정 등

4.2 Lucas-Kanade optical flow(Sparse Optical Flow)

- Optical Flow의 대표 알고리즘
- 아래 3가지 가정 하에 **추적하기 좋은 특징점(코너)**을 선택해서 그 점들만 추적
 - 밝기 보존 가정 : 같은 물체는 다음 프레임에서도 밝기가 거의 변하지 않음
 - 작은 이동 가정 : 한 프레임에서 다음 프레임까지의 움직임이 너무 크지 않음
 - 동일한 움직임 가정 : 작은 영역 안에서는 모든 픽셀이 동일한 방향으로 움직임
- 연산량이 적고 빠름, 작은 움직임에 강함(but 큰 움직임엔 약함)
- **Pyramidal 방식**으로 다중 해상도 추적 가능

[두 프레임 사이에서의 움직임(Optical Flow)을 추정, 물체의 이동 방향 시각화]

```
src1 = cv2.imread(Path(folder, "frame1.jpg")) # 첫 번째 프레임 (이전 프레임)
src2 = cv2.imread(Path(folder, "frame2.jpg")) # 두 번째 프레임 (현재 프레임)
```

```
# 전처리 - 그레이스케일(흑백이미지) 변환
gray1 = cv2.cvtColor(src1, cv2.COLOR_BGR2GRAY)
```

```
# Shi-Tomasi 알고리즘으로 추적할 좋은 코너(특징점)를 찾기
pt1 = cv2.goodFeaturesToTrack(gray1, 50, 0.01, 10)
→ 최대 50개의 특징점, 0.01: 최소 품질(quality level), 10: 최소 거리(픽셀 간 거리)
```

```
# Lucas-Kanade Optical Flow 계산
pt2, status, err = cv2.calcOpticalFlowPyrLK(src1, src2, pt1, None)
→ pt1: 첫 번째 프레임에서 추적할 점들, pt2: 두 번째 프레임에서의 추적된 위치
-> status: 추적 성공 여부(1: 성공, 0: 실패), err: 추정 오차
```

```
# 두 프레임 합성 (두 프레임을 50%씩 혼합 → 움직임이 보이기 쉬운 배경 생성)
dst = cv2.addWeighted(src1, 0.5, src2, 0.5, 0)
```



```

# 추적에 실패한 점은 건너뛴
for i in range(pt2.shape[0]):
    if status[i, 0] == 0:
        continue
# 첫 번째 프레임의 포인트를 노란색 원으로 그림
cv2.circle(dst, tuple(pt1[i, 0].astype(int)), 4, (0, 255, 255), 2, cv2.LINE_AA)
# 두 번째 프레임의 위치를 빨간색 원으로 표시
cv2.circle(dst, tuple(pt2[i, 0].astype(int)), 4, (0, 0, 255), 2, cv2.LINE_AA)
# pt1 → pt2로 화살표 그리기 → 모션 방향 벡터
cv2.arrowedLine(dst, tuple(pt1[i, 0].astype(int)), tuple(pt2[i, 0].astype(int)),
                 (0, 255, 0), 2)

```

4.3 Dense optical flow(Farneback 알고리즘)

- 모든 픽셀에 대해 이동 벡터를 계산
- 전체 영상 흐름 분석, 군집 움직임, 패턴 인식 등에 유용

[연속된 프레임 사이의 모든 픽셀의 움직임(방향 & 크기)을 추정하여 벡터 형태로 시각화]

Optical Flow 결과를 화살표로 시각화하는 함수

```
def draw_flow(img, flow, step=16):
```

```

    h, w = img.shape[:2] # x, y: step 간격으로 샘플링된 좌표 격자
    y, x = np.mgrid[step/2:h:step, step/2:w:step].reshape(2, -1).astype(int)
    fx, fy = flow[y, x].T # flow에서 각 포인트에 대한 x, y 방향 벡터(fx, fy) 추출
    # 각 점에 대해 시작점(x, y) → 끝점(x+fx, y+fy)으로 선분 생성
    lines = np.vstack([x, y, x+fx, y+fy]).T.reshape(-1, 2, 2)
    lines = np.int32(lines + 0.5) # .T.reshape(-1, 2, 2)는 (시작, 끝)좌표로 선 생성

```

```
# 흑백 이미지를 컬러로 변환 (시각화용 배경)
```

```
vis = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
```

```
# 움직임 벡터를 노란색 선 (화살표 없이) 으로 그리기
```

```
cv2.polylines(vis, lines, 0, (0, 255, 255), lineType=cv2.LINE_AA)
```

```
# 각 시작점에 주황색 점을 찍음 (움직임 시작 위치 표시)
```

```
for (x1, y1), (_, y2) in lines:
```

```
    cv2.circle(vis, (x1, y1), 1, (0, 128, 255), -1, lineType=cv2.LINE_AA)
```

```
# 시각화된 이미지를 반환
return vis
```

```
# 영상 파일 로드 (vtest.avi)
cap = cv2.VideoCapture(Path(folder, "vtest.avi"))
→ 참고) 실시간 웹캠 사용 시는 cv2.VideoCapture(0) 사용 가능
```

```
# 두 번째 프레임도 그레이스케일 변환 (Optical Flow는 흑백 기반)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)
# Dense Optical Flow 계산
flow = cv2.calcOpticalFlowFarneback(gray1, gray2, None, 0.5, 3,
                                     13, 3, 5, 1.1, 0)
```

→ 주요 파라미터 설명

0.5: 이미지 피라미드 스케일, 3: 피라미드 레벨 수, 13: 윈도우 크기, 3: 반복 횟수

5: 폴리널 계수 크기, 1.1: 폴리널 시그마, 0: 플래그(옵션 없음)

-> 결과 flow: H x W x 2 배열로 각 픽셀의 움직임 벡터 (dx, dy)를 저장

• Dense Optical Flow vs Sparse Optical Flow

비교 항목	Farneback(Dense)	Lucas-Kanade(Sparse)
픽셀 적용 범위	모든 픽셀	일부 특징점
계산량	많음	적음
표현력	풍부한 표현	경량화에 강점
시각화	벡터 필드	점 + 화살표
속도	느림	빠름
용도	전체 장면 분석	객체 추적

4.4 Mean Shift

• **개념:** 히스토그램(색 분포)의 중심이 어디로 이동해야 밀도가 높은지 계산하는 방식

• 특징

- 색 기반 추적에 효과적
- ROI 크기가 고정
- 회전에 대응하지 못함

- 조명 변화에 약함

[사용자 지정 ROI(관심 영역)를 히스토그램 기반으로 추적하는 Mean Shift 구현]

```
# 비디오 파일 열기 및 첫 프레임 읽기
cap = cv2.VideoCapture(Path(folder, "Billard.mp4"))
ret, frame = cap.read()
→ ret: 읽기 성공 여부(True/False), frame: 첫 번째 영상 프레임(이미지 형태)

# 사용자 ROI(추적 영역) 직접 지정
(x, y, w, h) = cv2.selectROI('ROI', frame) # 마우스로 사각형을 지정
rc = (x, y, w, h) # 반환값은 (x, y, w, h) → 좌상단 좌표(x, y) + 폭/높이
→ rc는 현재 추적 중인 영역을 의미

# 선택된 ROI를 HSV 색공간으로 변환
roi = frame[y:y+h, x:x+w] # roi는 선택된 영역의 이미지
roi_hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
→ HSV 공간으로 변환된 ROI
→ HSV는 조명 변화에 덜 민감하여 색상 기반 추적에 적합

# 히스토그램 계산(hist는 추적할 대상의 색 분포 정보)
channels = [0, 1] # H(색상) + S(채도) 채널 사용
ranges = [0, 180, 0, 256] # 각각의 채널 범위 설정
hist = cv2.calcHist([roi_hsv], channels, None, [90, 128], ranges)
→ H채널을 90개 bin, S채널을 128개 bin으로 나눔

# Mean Shift 알고리즘 종료 조건 설정
term_crit = (cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 1)
→ 10번 반복하거나, 중심점의 변화가 1픽셀 이하면 반복 종료

# 프레임 반복 처리 시작(비디오가 끝날 때까지 한 프레임씩 읽음)
while True:
    ret, frame = cap.read()
    if not ret:
        break
```

```

# 현재 프레임에서 히스토그램 역투영(Back Projection)
frame_hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
backproj = cv2.calcBackProject([frame_hsv], channels, hist, ranges, 1)
→ ROI 히스토그램(hist)과 비교하여, 유사한 색을 갖는 픽셀 위치에 높은 확률 부여
-> backproj: 확률 분포 영상 (흰색일수록 유사도 ↑)

# Mean Shift 알고리즘 적용
_, rc = cv2.meanShift(backproj, rc, term_crit) # backproj: 확률 이미지
# rc: 이전 추적 영역, 결과적으로 rc가 가장 유사한 색 분포로 이동

# 새로 갱신된 추적 영역을 빨간 사각형으로 표시
cv2.rectangle(frame, rc, (0, 0, 255), 2) # rc: 새로 계산된 ROI 위치
cv2.imshow('frame', frame) # 프레임을 윈도우에 띄움

# ESC 키(코드 27)를 누르면 루프 종료
if cv2.waitKey(20) == 27:
    break

# 비디오와 윈도우 리소스 정리
cap.release() # 비디오 객체 해제
cv2.destroyAllWindows() # 모든 OpenCV창 종료

```

4.5 Camshift(Continuously Adaptive MeanShift)

- MeanShift의 확장 버전

- 위치와 대상의 크기, 방향 변화도 자동으로 추적
- 객체 회전 대응, 결과가 회전된 타원형태로 반환
- 얼굴 인식, 손 추적 등에서 활용

```

# CamShift 적용(CamShift로 가장 유사한 위치로 ROI 이동)
ret, rc = cv2.CamShift(backproj, rc, term_crit)
→ ret: 타원의 중심, 크기, 회전각 포함 (RotatedRect)
-> rc: 다음 추적을 위한 사각형 위치 (업데이트됨)

```

결과 시각화

```
cv2.rectangle(frame, rc, (0, 0, 255), 2)
```

```
cv2.ellipse(frame, ret, (0, 255, 0), 2)
```

→ 빨간 사각형: ROI 사각 영역, 초록 타원: CamShift가 추정한 회전 및 크기 반영 영역

• CamShift vs MeanShift

기능	MeanShift	CamShift
위치 추적	O	O
크기 변화 대응	X	O
회전 대응	X	O
결과 반환	사각형	타원(RotatedRect)
사용 예시	고정된 물체, 단순 색 추적	얼굴, 손, 사람 등 유동적인 대상 추적

4.7 RAFT (Recurrent All-Pairs Field Transforms)

- **RAFT(Recurrent All-Pairs Field Transforms)**는 2020년에 발표된 최신 Optical Flow 추정 모델
- 기존 모델들과 비교하여 정확도와 일반화 성능 모두 매우 우수한 딥러닝 기반 방법
- 이름 그대로, 모든 픽셀 쌍의 관계를 반복적으로 계산하는 방식

<중요 개념>

- **All-Pairs Correlation Volume**
 - 두 이미지 간 모든 픽셀 쌍의 상관 관계를 계산
 - 일반적인 Optical Flow는 근처 픽셀만 비교
 - RAFT는 전체 픽셀쌍을 고려 → 훨씬 정밀한 계산 가능
- **Recurrent Update Module**
 - RAFT는 단번에 흐름을 예측하지 않고, GRU 기반 반복 구조로 optical flow를 점진적으로 정제
 - 예측 → 업데이트 → 예측 → ... 반복 수행
 - 이 구조 덕분에 고속 + 고정밀 가능

- **High-Resolution Flow**

- 흐름은 **원본 해상도**로 직접 계산됨(upsample 또는 coarse-to-fine 아님)
- 작은 물체도 잘 추적할 수 있음

<구조>

- **Feature Extractor**

- 두 입력 이미지를 각각 feature map으로 인코딩

- **Correlation Volume**

- 모든 feature 쌍의 dot product를 계산 → 거대한 상관 맵

- **Recurrent GRU**

- 초기 flow를 추정한 뒤, 여러 차례 반복 업데이트

- **Flow Prediction**

- 반복적으로 정제된 Optical Flow 출력
-

5. CNN기반 분류 앱

5.1 LeNet-5

- 개요

- 1998년 Yann LeCun이 제안한 초기 CNN 구조
- 손글씨 숫자 인식(MNIST)에 최적화
- 현재 기준 매우 작은 모델이지만, CNN 구조의 기초를 완성한 모델

- 특징

- Activation: 주로 **tanh**
- Pooling: **Average Pooling**
- 작은 필터와 얇은 구조
- 현재는 성능이 낮지만 CNN의 기본 흐름을 이해하는 데 중요

5.2 AlexNet

- 개요
 - 2012 ImageNet 우승
 - ReLU 사용, GPU 병렬 학습 도입
 - 대규모 CNN의 시대를 연 모델
- 전이학습(Transfer Learning) 방식
 - 사전학습된 AlexNet을 불러옴
 - 마지막 FC(Classifier)를 교체하여 새로운 데이터셋에 맞춤
 - 학습은 일반적으로
 - 특징 추출부는 동결(freeze)
 - 마지막 분류기 부분만 fine-tuning

```
model = models.alexnet(weights='IMAGENET1K_V1')
num_fts = model.classifier[6].in_features
model.classifier[6] = nn.Linear(num_fts, num_classes)
```

- 장점
 - 적은 데이터에서도 좋은 성능
 - 빠른 학습
 - 실무 앱 개발 시 가장 많이 활용되는 접근

5.3 VGGNET

- 특징
 - **3×3 Conv + 2×2 MaxPool** 반복 하는 규칙적이고 단순한 구조
 - 깊이가 깊어질수록 성능이 증가하며 표현력 강화
- VGG13 모델기반 로딩 및 추론 흐름
 1. Pretrained VGG13 모델 로드

```
model = models.vgg13(weights='IMAGENET1K_V1')
```

```
model.eval()
```

2. PIL → Tensor 변환
3. Normalize(mean,std) 적용
4. Softmax 확률로 변환
5. 가장 높은 확률의 클래스를 출력

5.4 GoogLeNet

- 개요
 - 다양한 크기의 필터(1×1 , 3×3 , 5×5)를 병렬로 적용하는 **Inception 모듈** 사용
 - 1×1 Conv를 통해 연산량 감소 → 경량 고성능 모델
- 병렬 구조 예시
 - 1×1 Conv
 - 3×3 Conv
 - 5×5 Conv
 - Max Pool → 1×1 Conv

이를 하나의 블록에서 병렬로 수행 → 특징 추출의 다중 스케일 확보
- 특징
 - 깊고 복잡한 구조
 - Global Average Pooling으로 FC 층 제거
 - 파라미터 수는 VGG 대비 훨씬 적음
 - 실제 응용에 적합한 효율적 모델

6. 신경망 스타일 전이(Neural Style Transfer)

6.1 Neural Style Transfer란?

- 한 이미지의 내용(Content)과 다른 이미지의 스타일(Style)을 결합해 새로운 이미지를 생성하는 기법

- Content image + Style image → 합성된 Generated image 생성
 - **Content image**: 사진처럼 객체 배치, 구조가 중요한 이미지
 - **Style image**: 그림처럼 색감, 붓터치, 질감이 중요한 이미지
 - 구조는 content처럼, 표현은 style처럼
 - 생성할 이미지를 하나의 학습 파라미터로 취급하여, Content Loss와 Style Loss를 최소화하도록 이미지 자체를 업데이트

6.2 Model Architecture

- NST는 사전 학습된 CNN(주로 VGG16/VGG19)을 기반으로 사용
- 고정된 Feature Extractor로 VGG16 사용 (가중치 업데이트 X)
- 중간 레이어에서 Content 및 Style Feature 추출
 - 낮은 레이어: Edge, corner
 - 중간 레이어: Texture, style pattern
 - 높은 레이어: 객체 구조, semantic 정보
- 새로운 이미지(generated image)를 학습하며, 두 특성을 동시에 맞춤
- 기존 CNN 구조를 입력 특징 추출용으로만 사용하고 생성 이미지가 학습대상으로서 최적화 되는 방식

6.3 Content Representation

- Content는 이미지의 구조적 특징(형태, 객체 배열)을 의미
 - CNN 특정 레이어의 feature map을 그대로 사용하여 비교
 - 일반적으로 VGG16의 conv4_2처럼 구조적 특징이 잘 추출되는 중간 레이어 사용
- Content Loss 공식

$$L_{content} = \|F_{content} - F_{generated}\|_2$$

6.4 Style Representation (Gram Matrix)

- Style은 이미지의 색감, 질감, 패턴, 즉 채널 간 상관관계를 모델링
- Gram Matrix란?

- Feature map 채널들 간의 **내적(dot product)** 관계를 담은 행렬
- 특정 스타일의 패턴 반복, 질감을 정량화하는 방식
- spatial 정보를 버리고 channel 간 상관관계만 남김 → 스타일 정보만 추출 가능
- Gram Matrix 공식

$$G = FFT$$

- Style Loss

$$L_{style} = l \sum w_l \|G_{l_{style}} - G_{l_{generated}}\|^2$$

6.5 NST의 전체 Loss 구조

- 최종 Loss는 다음 두 항의 조합

$$L_{total} = \alpha L_{content} + \beta L_{style}$$

- α (alpha): Content 비중
- β (beta): Style 비중
- β 가 크면 결과물이 더 화풍에 가까워짐

6.6 NST 학습 과정

1. 초기 Generated Image 설정
 - Content image 그대로 사용하거나
 - Random noise로 시작 가능 (빠르게 수렴하지는 않음)
2. Forward Pass
 - Content, Style, Generated image를 CNN에 통과시키고
 - 각 레이어에서 feature map과 Gram Matrix를 추출
3. Loss 계산
 - content loss
 - style loss

4. Backpropagation

- CNN 가중치가 아니라 생성된 이미지 tensor 자체에 대한 gradient를 계산

5. Optimizer로 이미지 업데이트

많이 사용되는 최적화 기법:

- L-BFGS (전통적 NST에서 가장 많이 사용)
 - Adam

6.7 Neural Style Transfer 코드

- 스타일 이미지와 콘텐츠 이미지를 PyTorch 텐서로 변환

```
# 이미지 파일 경로를 통해, (1, 3, H, W) 형태의 텐서(tensor)로 변환하는 함수
def image_to_tensor(image_filepath, image_dimension=128):
    # image_dimension: 크기 제한
    # PIL을 사용하여 이미지 로드, 흑백/투명 채널을 제거하고 항상 3채널(RGB)로 변환
    img = Image.open(image_filepath).convert('RGB')

    # resize 기준 설정: 최대 너비/높이를 기준으로 Resize할 크기를 설정
    if max(img.size) <= image_dimension:
        img_size = max(img.size)
    else:
        img_size = image_dimension

    torch_transformation = torchvision.transforms.Compose([
        torchvision.transforms.Resize(img_size), # 가장 큰 축을 img_size 리사이즈
        torchvision.transforms.ToTensor() # (H, W, C) → (C, H, W) 로 변환
    ])
    # .unsqueeze(0): 배치 차원 추가/ (3, H, W) → (1, 3, H, W)
    img = torch_transformation(img).unsqueeze(0)

    # device: GPU or CPU로 이동, dtype을 float으로 설정
    return img.to(device, torch.float)
```

- **Gram Matrix** 계산

- 스타일 이미지는 원본 이미지와 달리 색감·질감 등 **특정 레이어의 채널 간 상관관계**로 표현되며, 이를 정량화한 것이 Gram Matrix라고 볼 수 있음

```
# 입력된 CNN feature map 텐서에서 Gram Matrix를 계산
# 이 행렬은 NST에서 style loss 계산 시 사용
# 입력 텐서 ip는 shape = (N, C, H, W)
# N: 배치 크기, C: 채널 수, H, W: 세로, 가로 크기(feature map의 공간 크기)

def gram_matrix(ip):
    num_batch, num_channels, height, width = ip.size()
    # 텐서를 2D 행렬로 펼침(flatten)
    # -> 각 행은 하나의 채널에 해당하는 feature map을 벡터화, shape = (N × C, H × W)
    feats = ip.view(num_batch * num_channels, width * height)
    # feats × featsT (내적) → 채널 간 상관관계 행렬 계산, shape = (N × C, N × C)
    gram_mat = torch.mm(feats, feats.t()) # 각 원소 Gij는 채널 i와 채널 j 간의 유사도
    # Gram Matrix는 입력 크기에 따라 스케일이 달라지므로 정규화,
    # 나누는 값은 총 feature 수 = N × C × H × W
    return gram_mat.div(num_batch * num_channels * width * height)
→ 레이어마다 feature 크기가 달라도 균형 있게 style loss에 기여
```